

разница ansible/puppet/saltStack/chef

ANSIBLE:

- есть 100 серверов, нужно:
- поставить vim
- поправить sshd_config
- создать юзера devops

работает просто - по модели PUSH, то есть ты пошел, руками запустил плейбук или ad-hoc команду (ad-hoc - когда можно в терминале ввести одну команду и не писать целый плейбук, например:

```
ansible all -m yum -a "name=vim state=present"
```

и соотв. запустили любым способом и ансибл CAM сходит на все серваки по SSH и выполнит задачи

PUPPET:

предположим сценарий тот же

- на все сервера ставится puppet агент
- есть сервер - puppet master, на нем заводятся манифесты типа `package {'vim': ensure => installed}`
- `user {'devops': ensure => present}`

- условно каждые 30 минут сервер делает puppet agent --test и CAM приводит себя к этому состоянию

то есть даже если админ сам руками делитнет тот же вим, папИт вернет через полчаса на похуй все обратно

ТО ЕСТЬ ПАПИТ РАБОТАЕТ ПО МОДЕЛИ ПУЛЛ - КОГДА СЕРВАК ВСЕ ИЗМЕНЕНИЯ

SALTSTACK

- ансибл + папит в одном флаконе
- может работать как по пуш модели - то есть мы сами пушим изменения на серваки
- может работать и по пулл модели

CHEF - тот же папит, только по другому пишутся конфиги

КАКИЕ БЫВАЮТ АГЕНТЫ

1. АГЕНТЫ МОНИТОРИНГА

- zabbix агент - состояние сервера
- CPU, RAM, диск, сеть
- доступность портов
- шлет алерты

любят везде, особенно в банках, госухе, старых компах

- prometheus node exporter
- CPU, LA, RAM, сеть + куча всего другого
- работает по пулл модели, стандарт для кубера, девопса

2. АГЕНТЫ ЛОГИРОВАНИЯ

- filebeat (elastic stack)
- микроагент который отправляет логи
- читает логи с сервака, пакует строки, отправляет в elasticsearch, logstash, kafka

зачем нужен?

- логи вечно хуй пойми где лежат на серваках, а еще и серваков самих дохера, а мы хотим одно место, куда будут приходить все логи, плюс докидываем туда всякие плюшки типа фильтрации, парсинга и т.д.

сюда же идут такие ребята как: Logstash Forwarder / Beats / Fluentbit / Vector

- из интересного нам - fluentbit - мастхев для контейнеров в кубере, ставится сайдкарм

3. АГЕНТЫ ПРИЛОЖЕНИЙ

- стоят как бы внутри аппок, не на ОС

то есть это не агент, не бинарь, а БИБЛИОТЕКА + КОНФИГ ДЛЯ НЕЕ

питон - pip, node - npm, go - импорты в код и тд, то бишь эти агенты - куски кода

нужны, чтобы понимать:

- что тормозит
- сколько времени занимает запрос
- где ошибки
- как ведут себя сервисы
- как работает база

из популярных - sentry, dynatracem appdynamics, newrelic

4. АГЕНТЫ ИБ

- DLP АГЕНТЫ (DATA LOSS PREVENTION)

слежка за утечками

- кто копирует файлы
- кто качает базы
- кто юзает флешку
- куча всякого ибшного бреда

и может блочить операции, логировать все, слать в SOC

для нас важно, что он ставится также ансиблом и его аналогами, либо пакетом rpm/deb

- после установки подгружает конфиг с иб-сервера условного, либо уже зашит в образ ос, вm, контейнера

- АГЕНТЫ АУДИТА

в линуксе - auditd = вообще стандартный демон, ставится через папит тот же нужен для:

- кто вошел
- кто дал sudo
- кто меня конфиги в etc
- кто запускал команды

- EDR

улучшенный антивирус

- ловит майнеры, ловит брутфорс, блокирует бинарники, мониторит все что запущено

ставится тоже просто - скачивает пакет, добавляется systemd unit, выдаются серты/ключи, прописывается хост центрального ИБ сервака



физические сервера - bare metal

- железные ящики в стойках

ЖИЗНЕННЫЙ ЦИКЛ ДЕПЛОЯ НА СЕРВАКИ

1. закупка железа
 2. монтаж в стойку
 3. установка базового ПО
 - на чистый сервак ставится
 - линукс / винда
 - RAID/диски
 - базфвые пакеты
 - агенты мониторинга / логирования
 - фаерволы, все что связано с безопасностью
- окей, вот мы настроили сервак, каким образом деплоятся приложения, если не в контейнерах?

- ТАКЖЕ
1. сначала нужно поставить окружение для языка
 - java JDK
 - python + virtualenv
 - node.js - npm
 - go
 - .net runtime
 2. поставить все зависимости
 - всякие libssl, libxml
 - curl
 - jq
 - net-tools
 3. дальше нужно подкинуть сертификаты TLS/внутренние CA (центры сертификации)
 - скопировать куда надо
 - обновить хранилище
 - настроить серты для nginx, java, питон и тд

4. нужно залить конфиги в нужные папки

на баре металл обычная структура такая

```
/opt/app/  
/opt/app/releases/  
/opt/app/current/  
/opt/app/config/  
/opt/app/logs/
```

releases — версии приложения (app-v1.2.3.jar)
current — симлинк на текущую версию
config — конфиги
logs — логи

- проставить переменные окружения, секреты, адреса сервисов для интеграции

5. также примерно всегда создается пользак под сервис (условно appuser, service)
- ему выдаются права на папки, создается группа deploy, ops, monitoring
6. настройка фаерволла, открыты нужные порты, всякие SELinux и тд
7. ну и что-то сделать с сеткой

ОКЕЙ, ЭТО ВСЕ ЕЩЕ ПОДГОТОВКА, КАК ДЕПЛОИТЬ? (ТАК НАЗЫВАЕМЫЙ ПРОЦЕСС DEPLOYMENT)

1 ВАРИК

- scp/rsync + systemd - самый легаси, ебланский способ
1. приложение лежит в условной папке - /opt/app/current
 2. приложение нужно как-то собрать - либо локально где-то, либо все таки есть уже CI
 3. собрали - получили артефакт (jar, war, zip, бинарь)
 4. копируем на сервер это дело самостоятельно (scp/rsync/ftp не дай бог)
 5. перекладываем в /opt/app/releases
 6. меняем симлинк /opt/app/current на новую версию
 7. перезапускаем systemd - systemctl restart app
 8. смотрим как обезьяны в логи
 9. если поломалось - обратно меняем симлинк

2 ВАРИК

ansible деплой - это не CI/CD упаси, это просто автоматизация процесса выше

- все что описано выше - автоматизируется ролями, плейбуками, заводятся переменные, конфиги, темплейты, сюда же хелзчеки, и все делается по кайфу, уже лучше, но еще не контейнеры

3 ВАРИК

тема про которую я вот только щас узнал - OCTOPUS DEPLOY

- это какая-то огромная корп хуетень, которая типа управляет не деплоем, а релизами

ЭТО ВСЕ ХОРОШО, НО ТАК МАЛО ГДЕ ПРОИСХОДИТ, ПОЭТОМУ ВОТ ГДЕ ОБЫЧНО ЮЗАЮТ БАРРЕEEEEEEEEEE МЕТАЛ

вообще как вы поняли наверное, баре метал нужен там где высокая нагрузка, нужны гарантии для иб, нельзя терять пакеты в сетке, нужны всякие лиуцхи, огромные базы, кластеры, и т.д

где все-таки используют?

- брокеры, биржи
- банки (ядро)
- высоконагруженные бд (oracle, db2)
- кафка кластеры

1) можно автоматизировать PXE - загрузка ОС по сети - в биосе стоит Boot from network (PXE)

используется:

- Kickstart
- Preseed

для винды на всякий - WDS (Windows Deployment Services)

2) если не автоматизация, то через ISO образ, но можно не ходить в ЦОД, а подключиться к голой консолюке сервака, подсунуть ему ISO образ, грузануть установщик и прокликать инсталлятор

практически не автоматизируется

настраивается через

- BIOS/UEFI
- руками челика, сидящего в ЦОДе
- спец. утилиткой производителя железа

есть еще темка - CLI RAID tools - но это полуавтомат и все равно нужен микрочел

автоматизируется всегда (80-100%)

чен?

- ansible
- puppet
- salt
- chef (редко)
- и не забываем про баш-скрипты

автоматизируется всегда

ставится ребятами из блока слева